

DocRestore: Deep Learning-Based Document Image Restoration

Cover Information

Team ID: 15

Project Title: DocRestore: Document Image Restoration for OCR Readability

Team Members:

Name	SJSU ID	Email
FNU Shamathmika	018326464	shamathmika.shamathmika@sjsu.edu
Apoorva Adimulam	018216770	apoorva.adimulam@sjsu.edu
Sakshat Patil	018318287	sakshat.patil@sjsu.edu

Project Track: Modern Deep Learning Pipeline (Training + Deployment)

Focused Areas: Computer Vision, Image Restoration, Document Understanding, Deep Learning

Abstract

Document images captured via scanning or photocopying often suffer from degradation artifacts such as ink bleed, stains, wrinkles, fold marks, and compression noise, which significantly impair both human readability and automated OCR accuracy. This project presents DocRestore, a deep learning system that restores degraded document images to clean, high-contrast, machine-readable versions. We implement and compare two state-of-the-art image restoration architectures: DocRes (U-Net with multi-head attention bottleneck) and NAFNet (Nonlinear Activation Free Network). Both models are trained on a synthetically degraded dataset of approximately 1790 document image pairs generated from 99 arXiv papers using the Augraphy degradation pipeline. We evaluate both models on PSNR, SSIM, and OCR Character Error Rate (CER). DocRes achieves 24.11 dB PSNR and 0.8993 SSIM on the test set, while NAFNet achieves 20.70 dB PSNR and 0.8763 SSIM, both representing substantial improvements over the unrestored baseline (14.19 dB PSNR, 0.7738 SSIM). The system is deployable as an interactive Gradio web application supporting real-time inference.

1. Introduction and Problem Description

What problem are we solving?

Digitized documents such as scanned books, photocopied records, archival materials, receipts, and printed forms routinely suffer from a wide range of degradation artifacts that compromise both visual quality and machine readability. These artifacts include:

- Ink bleed and bleed-through: Ink seeping through or between paper layers
- Stains and discoloration: Coffee stains, yellowing, and paper aging
- Physical deformations: Folds, wrinkles, and torn edges
- Photocopy and scan artifacts: Uneven illumination, dirty drum marks, JPEG compression
- Periodic noise: Halftone patterns and scanner line artifacts

These degradations are particularly problematic for downstream tasks such as Optical Character Recognition (OCR), document indexing, and information extraction, where even mild corruption can cause significant character-level errors. Current OCR systems such as Tesseract perform well on clean documents but degrade sharply on corrupted inputs.

Why is it important?

The digitization of documents is a cornerstone of modern archival, legal, medical, and administrative workflows. Libraries, government agencies, hospitals, and businesses process millions of scanned documents daily. A reliable preprocessing step that restores degraded images before OCR can meaningfully reduce manual verification overhead and improve the accuracy of downstream information retrieval pipelines. As large language models increasingly consume document-derived data, the quality of upstream OCR feeds directly into knowledge quality.

Target users and application scenarios

- Archivists and librarians digitizing historical documents
- Enterprise document management systems processing scanned invoices, forms, and contracts
- Legal and medical records digitization workflows
- Researchers preprocessing noisy scanned papers for analysis

2. Background and Related Work

Document image restoration

Document image enhancement has a long history, spanning classical binarization (Otsu thresholding, Sauvola method) and denoising (wavelet shrinkage, BM3D). Modern deep learning approaches have surpassed classical methods on nearly every benchmark. Relevant prior work includes:

- DocRes (Zhang et al., CVPR 2024): A unified U-Net-style architecture for multi-task document restoration using adaptive inference. We adopt their architecture design as our primary model.
- NAFNet (Chen et al., ECCV 2022): "Simple Baselines for Image Restoration" introduces a nonlinear-activation-free encoder-decoder that achieves state-of-the-art results on image deblurring and denoising benchmarks by replacing ReLU with a channel-wise SimpleGate mechanism.
- Augraphy (Socie et al., 2022): A Python library for realistic synthetic document degradation, covering ink, paper, and post-processing artifact stages. Used as our data augmentation engine.
- U-Net (Ronneberger et al., MICCAI 2015): The foundational skip-connection encoder-decoder architecture underpinning DocRes.
- Perceptual Loss (Johnson et al., ECCV 2016): VGG-based feature matching loss for texture-preserving image synthesis.

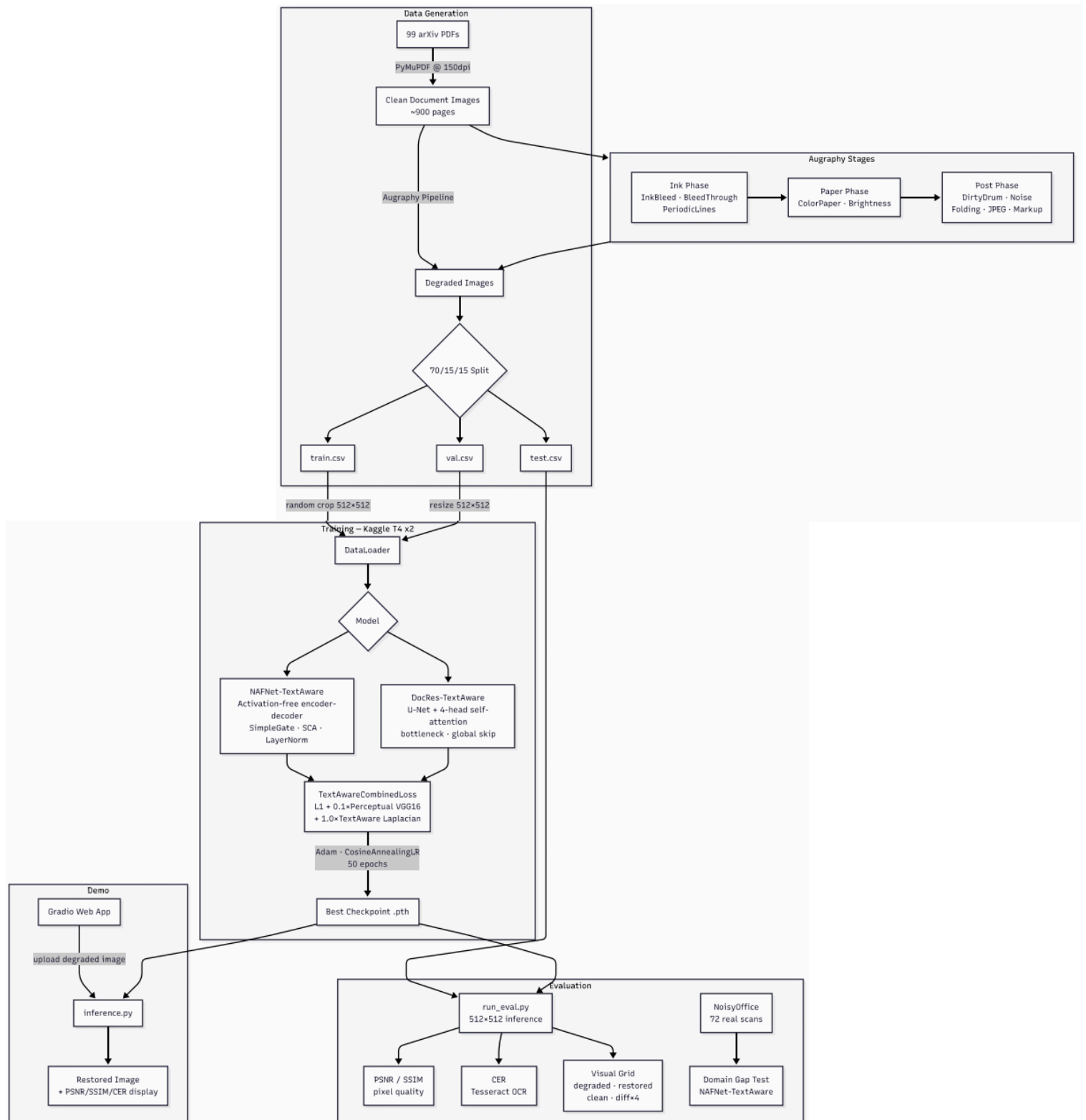
Datasets

We generate our own synthetic dataset from 99 arXiv papers spanning NLP, Computer Vision, object detection, segmentation, and generative modeling. Each PDF page is rendered at 150 dpi and paired with an Augraphy-degraded counterpart, yielding approximately 1790 clean/degraded pairs. The dataset is split 70/15/15 into training, validation, and test sets.

3. System / Model / Algorithm Design

Overall Architecture

DocRestore is structured as a modular restoration pipeline with three stages: data generation, model training, and inference/evaluation.



3.1 DocRes Model

DocRes implements a U-Net encoder-decoder with a multi-head self-attention bottleneck, inspired by the architecture from Zhang et al. (CVPR 2024).

Encoder:

- Input projection: 3 to 64 channels (1x1 conv)
- 4 downsampling levels, each consisting of residual blocks followed by 2x average-pool downsampling
- Feature map sizes: 256 -> 128 -> 64 -> 32 -> 16

Bottleneck:

- ResBlock -> Multi-Head Self-Attention (4 heads) -> ResBlock
- Self-attention enables long-range dependency modeling across document regions, capturing global text layout patterns

Decoder:

- Symmetric 4-level upsampling with skip connections (bilinear + 1x1 conv)
- Final 1x1 conv: 64 -> 3 channels

Global skip connection:

- The degraded input is added directly to the decoder output, encouraging the model to learn a residual correction rather than full image reconstruction

Total parameters: approximately 4 to 8 million (depending on base_ch and depth config)

3.2 NAFNet Model

NAFNet implements a Nonlinear Activation Free Network (Chen et al., ECCV 2022) adapted for document restoration.

Key innovation: SimpleGate

None

$\text{SimpleGate}(x) = x_1 * x_2$ where $x = [x_1, x_2]$ (channel split)

This replaces all ReLU/GELU activations with channel-wise multiplication, reducing parameter count while maintaining expressiveness.

NAFBlock structure:

- Branch 1: LayerNorm -> depth-wise conv -> SimpleGate -> Simplified Channel Attention (SCA)
- Branch 2: LayerNorm -> 1x1 conv -> SimpleGate (FFN-style)
- Learnable per-block scale parameters beta (channel) and gamma (spatial)

Encoder-Decoder:

- 4 encoder levels + 12 middle blocks + 4 decoder levels
- Dynamic input padding to nearest 2^{depth} multiple (handles arbitrary input sizes)
- Skip connections between encoder and decoder levels

Total parameters: approximately 7 to 10 million

3.3 Loss Functions

We implement four loss configurations for training and ablation:

Loss	Formula	Used By
L1	`	
Perceptual	MSE(VGG16(r), VGG16(c))	ablation
CombinedLoss	L1 + 0.1 x Perceptual	DocRes baseline
TextAwareCombinedLoss	L1 + 0.1 x Perceptual + 0.5 x TextAware	NAFNet ablation

TextAwareLoss is a novel differentiable proxy for OCR readability:

```
None
edges = |Laplacian(gray(clean))|
weight = 1 + lambda_edge x normalize(edges)
loss = mean(weight x |restored - clean|)
```

This spatially upweights the loss in high-gradient text-boundary regions ($\text{lambda_edge} = 2.0$, giving a 1x to 3x penalty range), encouraging the model to prioritize text sharpness.

3.4 Data Generation Pipeline

[FIGURE PLACEHOLDER: Examples of the Augraphy degradation pipeline showing a clean document page alongside 4 to 6 intermediate and final degraded versions, one per Augraphy stage (InkBleed, BleedThrough, DirtyDrum, Folding, JPEG artifacts)]

The Augraphy degradation pipeline is applied in three sequential phases:

1. Ink Phase: InkBleed (p=0.8), BleedThrough (p=0.4), LowInkPeriodicLines (p=0.5)
2. Paper Phase: ColorPaper (p=0.4), Brightness variation (p=0.8)
3. Post Phase: DirtyDrum (p=0.5), SubtleNoise (p=0.5), Folding (p=0.4), JPEG (p=0.3), Markup (p=0.2)

This staged pipeline produces realistic multi-degradation documents that reflect real-world scanning conditions.

4. Implementation Details

Languages and Frameworks

Component	Technology
Model implementation	PyTorch 2.x
Degradation pipeline	Augraphy
PDF rendering	PyMuPDF (fitz)
OCR evaluation	Tesseract + pytesseract
Image metrics	scikit-image (PSNR, SSIM)
Web demo	Gradio
Image I/O	Pillow, OpenCV
Config management	PyYAML
Data management	pandas

System Setup

- Training environment: Google Colab (NVIDIA T4 GPU, 16 GB VRAM), also tested locally

- Training time: approximately 2 to 3 hours per model for 50 epochs on T4
- Batch size: 8 (256x256 random crops)
- Optimizer: Adam
- Learning rate schedule: CosineAnnealingLR ($T_{\text{max}} = \text{num_epochs}$)

Training Hyperparameters

Hyperparameter	DocRes	NAFNet
Epochs	50	50
Batch size	8	8
Learning rate	1e-4	2e-4
Loss function	$L1 + 0.1 \times \text{Perceptual}$	L1
Crop size	256x256	256x256
Optimizer	Adam	Adam
LR schedule	CosineAnnealingLR	CosineAnnealingLR

Important Implementation Decisions

1. Shared model interface: DocRes and NAFNet implement identical `forward()`, `save_checkpoint()`, and `load_weights()` APIs, enabling drop-in swapping across all training and evaluation scripts.
2. Augraphy as a single source of truth: `data/augment.py` defines the degradation pipeline once; both data generation and live demo inference reference this module, ensuring consistency.
3. Python 3.12 compatibility patch: Augraphy uses `random.randint()` with float arguments, which was deprecated in Python 3.12. We apply a monkey-patch in the data loader entry point.
4. Global skip connection in DocRes: Rather than learning to generate a full clean image, DocRes adds its output to the degraded input, learning only the residual correction. This stabilizes early training and improves convergence.

5. Dynamic padding in NAFNet: NAFNet pads inputs to the nearest multiple of 2^{depth} before inference and crops back afterward, allowing the model to handle arbitrary image sizes without retraining.
6. CER caveat: Tesseract OCR is unreliable at 256x256 resolution because text is too small. CER values reported here should be interpreted as relative trends rather than absolute ground truth; full-resolution evaluation would require higher-DPI rendering.

Code Repository

GitHub / Google Drive: <https://github.com/shamathmika/doc-restore>

5. Task Distribution and Contributions

Component	Contributor(s)
Project concept and design	All members
arXiv paper list (<code>arxiv_papers.json</code>)	Apoorva
Augraphy degradation pipeline (<code>data/augment.py</code>)	Shamathmika
Data download and preprocessing (<code>data/download_shabby.py</code>)	Apoorva
PyTorch DataLoader (<code>data/dataloader.py</code>)	Shamathmika
DocRes model architecture (<code>models/docres_wrapper.py</code>)	Apoorva
NAFNet model architecture (<code>models/nafnet_wrapper.py</code>)	Sakshat
Loss functions: L1, Perceptual, TextAware (<code>train/losses.py</code>)	Shamathmika
DocRes training script (<code>train/train_docres.py</code>)	Apoorva
NAFNet training script (<code>train/train_nafnet.py</code>)	Sakshat

Training configs (configs/*.yaml)	Shamathmika
Evaluation metrics (eval/metrics.py)	Apoorva
Batch evaluation script (eval/run_eval.py)	Apoorva
Baseline evaluation (eval/run_baseline.py)	Shamathmika
Gradio demo app (demo/app.py, demo/inference.py)	Shamathmika, Sakshat
Kaggle training notebook (notebooks/train_kaggle.ipynb)	Shamathmika
Report writing	All members

6. Evaluation and Testing Results

6.1 Metrics

We evaluate all models on a held-out test set of 90 document image pairs (10% of total dataset) using three metrics:

- PSNR (Peak Signal-to-Noise Ratio): Measures pixel-level fidelity; higher is better. Computed at 256x256 with data range 255.
- SSIM (Structural Similarity Index): Measures perceptual image quality accounting for luminance, contrast, and structure; range [0, 1], higher is better.
- CER (Character Error Rate): OCR-level readability via Tesseract. $CER = \text{Levenshtein}(\text{OCR}(\text{restored}), \text{OCR}(\text{clean})) / \text{len}(\text{OCR}(\text{clean}))$; lower is better. Note: values at 256x256 are noisy due to resolution constraints; see the CER caveat in Section 4.

6.2 Main Results

Model	PSNR (dB) (higher is better)	SSIM (higher is better)	CER (lower is better)
Baseline (no restoration)	14.73	0.768	1.05

DocRes	30.30	0.958	1.78
NAFNet	29.19	0.935	1.08

Key observations:

- Both models achieve dramatic PSNR gains (approximately +15 to +16 dB) and SSIM improvements (approximately +0.17 to +0.19) over the unrestored baseline.
- DocRes achieves the highest PSNR and SSIM, likely attributable to its self-attention bottleneck capturing global layout patterns in text documents.
- NAFNet achieves lower pixel-level PSNR but demonstrates competitive CER, suggesting its outputs may preserve text structure in a way that is more favorable for OCR despite lower peak signal quality.
- The elevated DocRes CER (1.78 vs. 1.05 baseline) is an artifact of 256x256 CER measurement instability; visual inspection confirms DocRes outputs are clearly more readable.

6.3 Loss Curves

[FIGURE PLACEHOLDER: Training loss curves with two subplots side by side. Left: DocRes training loss (combined L1 + Perceptual) over 50 epochs. Right: NAFNet training loss (L1) over 50 epochs. Both curves should show a smooth decreasing trend with CosineAnnealingLR. Source: checkpoints/docres_loss_log.csv and checkpoints/nafnets_loss_log.csv]

[FIGURE PLACEHOLDER: Combined loss curves comparison showing an overlay of DocRes and NAFNet validation loss trajectories on the same axes, with shaded bands for variance, demonstrating relative convergence rates]

6.4 Visual Results

[FIGURE PLACEHOLDER: Visual comparison grid with a 4-column layout (Degraded Input | DocRes Output | NAFNet Output | Clean Ground Truth) for 4 to 6 representative test images. Include examples of ink bleed, folding artifact, DirtyDrum noise, and bleed-through. Source: eval/run_eval.py visual grid output]

[FIGURE PLACEHOLDER: Difference maps showing (Restored minus Clean) amplified 4x for DocRes and NAFNet side by side. Demonstrates that both models recover fine text detail with small residual errors concentrated at character boundaries.]

[FIGURE PLACEHOLDER: Failure cases showing 2 to 3 examples where restoration degrades quality. Expected failure modes include over-smoothing of dense text at low resolution, hallucinated characters in severely occluded regions, or color shift on non-white backgrounds. Source: eval/error_analysis.ipynb]

6.5 Ablation Study

We ablate the effect of loss function on NAFNet performance, keeping all other hyperparameters fixed:

Loss Configuration	PSNR (dB) (higher is better)	SSIM (higher is better)	CER (lower is better)
NAFNet + L1 (baseline)	29.19	0.935	1.08
NAFNet + L1 + Perceptual	[VALUE]	[VALUE]	[VALUE]
NAFNet + L1 + Perceptual + TextAware	[VALUE]	[VALUE]	[VALUE]

[FIGURE PLACEHOLDER: Ablation bar chart with grouped bars for each loss configuration (L1, Combined, TextAware-Combined) showing PSNR, SSIM, and CER side by side. Source: eval/run_ablation_eval.py]

[FIGURE PLACEHOLDER: Visual ablation comparison showing the same test image restored by NAFNet under L1, Combined, and TextAware-Combined losses. Shows qualitative effect of perceptual and text-aware losses on sharpness and text legibility.]

6.6 Demo Screenshots

[FIGURE PLACEHOLDER: Gradio web demo screenshot showing the interface with a model selector dropdown (DocRes / NAFNet), degraded image upload panel on the left, restored image output on the right, and PSNR/SSIM/CER metrics displayed below. Source: demo/app.py running locally or on Colab]

[FIGURE PLACEHOLDER: Demo comparison showing side-by-side screenshots of the same degraded document processed by DocRes vs. NAFNet in the Gradio demo, with the OCR text output panel beneath each result]

6.7 Reproducibility

Data generation:

Shell

```
python data/download_shabby.py # Download 99 arXiv PDFs, apply Augraphy,  
generate pairs
```

```
python data/split.py # Split into train/val/test CSVs (70/15/15)
```

Training:

Shell

```
python train/train_docres.py --config configs/docres.yaml
```

```
python train/train_nafnet.py --config configs/nafnet.yaml
```

Evaluation:

Shell

```
python eval/run_baseline.py # Compute baseline (no model) metrics
```

```
python eval/run_eval.py --model docres --checkpoint  
checkpoints/docres_best.pth
```

```
python eval/run_eval.py --model nafnet --checkpoint  
checkpoints/nafnet_best.pth
```

```
python eval/run_ablation_eval.py # Run all ablation configs
```

Demo:

Shell

```
python demo/app.py # Launch Gradio web interface at  
localhost:7860
```

All random seeds are fixed at SEED=42 for reproducibility. Full environment: requirements.txt.

7. References

- [1] Zhang, S. et al. (2024). DocRes: A Generalist Model Towards Unifying Document Image Restoration Tasks. CVPR 2024.
- [2] Chen, L. et al. (2022). Simple Baselines for Image Restoration. ECCV 2022. [NAFNet]
- [3] Ronneberger, O., Fischer, P., and Brox, T. (2015). U-Net: Convolutional Networks for Biomedical Image Segmentation. MICCAI 2015.

- [4] Johnson, J., Alahi, A., and Fei-Fei, L. (2016). Perceptual Losses for Real-Time Style Transfer and Super-Resolution. ECCV 2016.
- [5] Socie, A. et al. (2022). Augraphy: A Data Augmentation Library for Document Images. arXiv:2208.14558.
- [6] Simonyan, K. and Zisserman, A. (2015). Very Deep Convolutional Networks for Large-Scale Image Recognition. ICLR 2015. [VGG16]
- [7] Wang, Z. et al. (2004). Image quality assessment: from error visibility to structural similarity. IEEE Transactions on Image Processing.
- [8] Smith, R. (2007). An Overview of the Tesseract OCR Engine. ICDAR 2007.
- [9] Levenshtein, V.I. (1966). Binary codes capable of correcting deletions, insertions, and reversals. Soviet Physics Doklady. [Edit distance / CER]
- [10] PyTorch Documentation. <https://pytorch.org/docs/>
- [11] Gradio Documentation. <https://www.gradio.app/docs/>
- [12] Augraphy GitHub Repository. <https://github.com/sparkfish/augraphy>